

Exercice 1 :**Définitions**

On définit le type `'a arbre_bin`, correspondant aux arbres binaires, ainsi :

type `'a arbre_bin = Vide | Noeud of 'a arbre_bin * 'a * 'a arbre_bin`

On remarque qu'ici l'étiquette d'un arbre se situe entre les deux sous-arbres (choix différent de celui fait en cours). Ceci change évidemment l'écriture des fonctions, mais n'a aucune conséquence sur les algorithmes utilisés ou leur complexité.

1. Écrire des fonctions correspondant à :

- la construction d'un nouvel arbre binaire étant donné ses sous-arbres gauche et droit et son étiquette ;
- le calcul de l'étiquette d'un arbre binaire non vide ;
- le calcul du sous-arbre gauche d'un arbre binaire non vide ;
- le calcul du sous-arbre droit d'un arbre binaire non vide.

Remarque : Dans la suite il n'est pas pertinent d'utiliser ces fonctions. Ceci sera en général plus lourd que d'utiliser directement la syntaxe OCaml qui a permis d'écrire ces fonctions, de même qu'on utilise pas ou très peu les fonctions `List.hd` et `List.tl` sur les listes.

2. (a) Écrire une fonction `taille : 'a arbre_bin -> int` qui prend en argument un arbre binaire et renvoie son nombre de nœuds (c'est à dire sa taille).
- (b) Montrer par induction structurelle que si A est un arbre binaire de taille n , alors le constructeur `Vide` apparaît $n + 1$ fois dans A .
- (c) En déduire le nombre d'appels de la fonction `taille` lorsqu'elle est appelée sur A en fonction de sa taille.
- En déduire sa complexité dans le pire cas en fonction de la taille de l'argument, puis en fonction de sa hauteur.
3. Écrire une fonction `hauteur : 'a arbre_bin -> int` qui prend en argument un arbre binaire et renvoie sa hauteur.

Exercice 2 :**Nombre d'arbres binaires de taille n**

Dans tout cet exercice, on restreint l'ensemble des étiquettes au singleton $\{()\}$, donc on se place dans les `unit arbre_bin`.

Ceci revient à considérer les arbres comme non étiquetés (puisque'il n'y a pas de choix possible pour l'étiquette).

1. Énumérer les arbres binaires non étiquetés avec 0, 1, 2 ou 3 nœuds.
2. On note C_n le nombre d'arbres binaires non étiquetés de taille n . Établir une relation de récurrence sur C_n .

Remarque : C_n est le n -ème nombre de Catalan. On a $\forall n \geq 0, C_n = \binom{2n}{n} - \binom{2n}{n+1} = \frac{1}{n+1} \binom{2n}{n}$. Cependant, la preuve de la formule du terme général dépasse le cadre de ce TD.

3. Écrire une fonction `rand_bintree : int -> unit arbre_bin` qui prend un entier n et génère de façon aléatoire un arbre binaire non étiqueté de taille n . On ne demande pas un tirage uniforme¹, mais il faut que chaque arbre binaire de taille n ait une chance d'être généré.

On rappelle que la fonction `Random.int : int -> int` génère un entier aléatoire entre 0 (inclus) et son paramètre (exclu).

1. Comme on le verra dans le chapitre sur les arbres binaires de recherche, on obtient la même loi de probabilité sur les arbres (aux étiquettes près) si on ajoute dans un arbre binaire de recherche (qui n'est pas auto-équilibrant) des réels pris de façon indépendante et uniformément aléatoire dans un intervalle.

Exercice 3 :**Dans les profondeurs**

La profondeur d'un nœud N dans un arbre A est définie ainsi :

- Si N est la racine de A , sa profondeur dans A est 0.
- Sinon, N admet un unique parent M dans A . La profondeur de N est 1 plus la profondeur de M dans A .

1. On cherche à écrire une fonction `plus_profond : 'a arbre_bin -> 'a` qui renvoie l'étiquette d'un des nœuds (choisi arbitrairement) les plus profonds d'un arbre binaire non vide.

Si l'arbre est vide, on soulèvera une exception avec `invalid_arg`.

- (a) Donner une suite d'arbres $(A_n)_{n \in \mathbb{N}}$ où A_n est de taille n qui montrent que la fonction ci-dessous a une complexité quadratique en la taille de l'arbre dans le pire cas.

```
1 let rec plus_profond a = match a with
2   | Vide -> invalid_arg "Arbre vide"
3   | Noeud (Vide, x, Vide) -> x
4   | Noeud (gauche, _, droit) ->
5     if hauteur gauche > hauteur droit then plus_profond gauche
6     else plus_profond droit
```

- (b) En utilisant une fonction auxiliaire qui calcule à la fois la hauteur et l'étiquette du nœud le plus profond, écrire une version de la fonction `plus_profond` de complexité linéaire.
2. (a) Écrire une fonction `profondeur_max : 'a arbre_bin -> 'a -> int option` qui prend en argument un arbre binaire a et une valeur x , et renvoie sous forme d'option (avec le constructeur `Some`) la plus grande profondeur à laquelle x apparaît dans a , ou bien `None` si x n'apparaît pas dans a .
- (b) Prouver la correction de votre fonction par induction structurelle.
3. Faire de même avec la plus petite profondeur à laquelle x apparaît dans a (fonction `profondeur_min`).